

Security Automation & Architecture Portfolio

Portfolio at a glance

- Scale: ~288k LOC
- Languages used: PowerShell ~249k LOC; Python ~8.6k LOC; C++ ~15.4k LOC; Shell/Batch ~1.3k LOC; HTML/CSS/JavaScript ~3.9k LOC; Markdown/Config/CSV ~8.9k LOC
- Language-agnostic delivery: I heavily leverage agentic AI to write and maintain code with tests and reviews.

RMM Platform (Full Stack)

- Web platform: centralized operations for firmware/config drift, host telemetry, and secrets via a cloud secrets manager; SSH/SFTP orchestration; staged firmware uploads.
- API bridge: platform API integrations for inventory/maintenance; normalized payloads; relational upserts for analytics/dashboards.
- Device manager: admin UI + management commands for config updates, credential rotation, secret exports; telemetry for capacity planning.
- Operational tooling: deployment helpers (release snapshots, update pulls, log pruning, worker/service restarts).
- Implementation: Django/Python/PostgreSQL on a single optimized Ubuntu host.

Core capability areas

- Incident response & containment (Windows Filtering Platform-based isolation).
- Detection engineering (high-signal rules, dashboards, replay/testing harnesses).
- Compliance automation (~500+ CIS-aligned checks; phased deployment; evidence/attestation exports).
- Forensics & timelines (multi-source artifacts; interactive HTML).
- RMM/MDM-agnostic deployment (universal wrappers; staged rollouts; health checks; self-healing).
- Security tool monitoring (multi-vendor health validation; registry/state exports).
- Threat intelligence workflows (IOC reputation checks; hunt exports).

Representative utilities (generic names)

- net-isolate (containment), triage-lite/full (evidence capture), ioc-hunt (IOC sweeps), timeline-build (forensics), xdr-rules-host (detections).
- cis-pack (control checks & evidence), deploy-universal (portable deployment), tool-monitor (health), update-repair (OS remediation).

Engineering practices

- Defensive-only intent; alignment with CIS Controls, NIST CSF, and Windows Security Baselines.
- Native API usage (Windows Filtering Platform, WMI/CIM, Registry, Event Logs); least-privilege execution; atomic rollback for critical operations.
- Robust error handling with detailed logging and recovery; modular architecture and reusable helpers.
- Tests and staged rollouts (unit/Pester where applicable), replay harnesses, performance/stability thresholds.

Operational outcomes (representative)

- Incident reduction via preventive controls and higher-fidelity detections.
- Faster incident response through one-click containment and standardized triage/timelines.
- Accelerated audits through automated evidence and control status reporting.
- Resilience and portability through vendor-neutral, RMM/MDM-agnostic deployment.